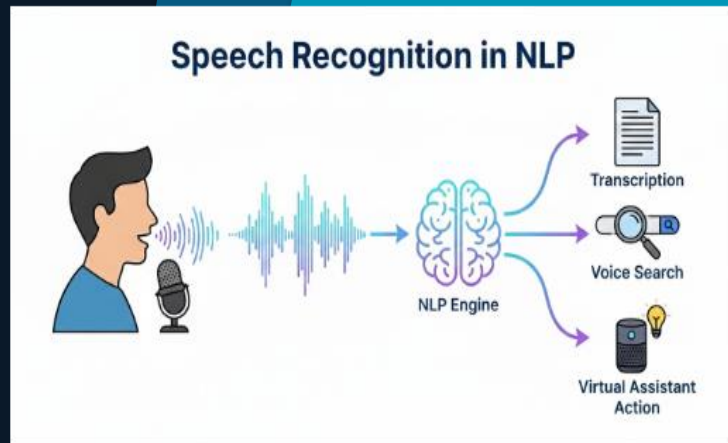


WEEK 8

Speech Recognition

as a Perception Module

From waveform features to AI pipeline integration



Audio

Features

ASR Model

Transcript

AI Pipeline

Why Speech Recognition Matters in AI

Most Natural Input

Speech is how humans naturally communicate. Giving machines an 'ear' makes interaction effortless and inclusive.

Unstructured → Structured

ASR converts raw audio into text — structured data that downstream reasoning modules can process and act on.

Enabling Downstream AI

Once transcribed, speech feeds search, dialogue systems, summarization, automation, and agent pipelines.

AI System Stack

Perception (ASR)

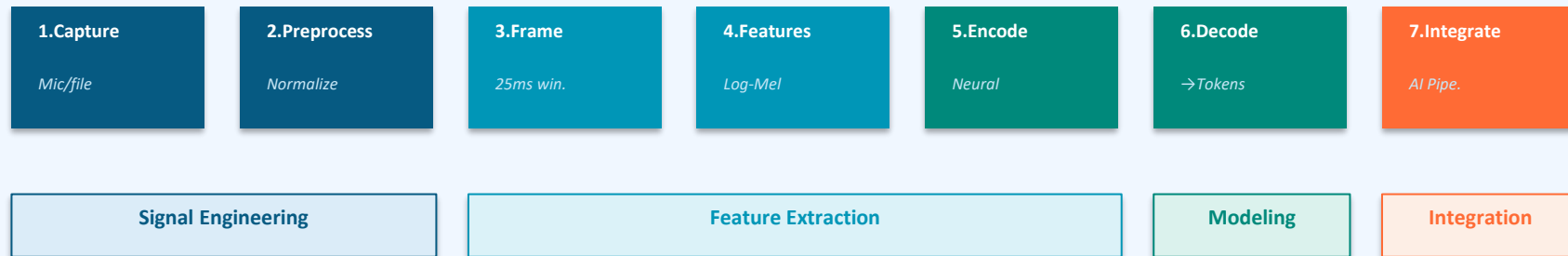
Understanding (NLU/LLM)

Decision Making

Action / Response

ASR sits at the base — it enables everything above it

End-to-End Speech Recognition Pipeline



- Audio enters as raw samples — preprocessing stabilizes the signal for analysis
- Feature extraction is the core bridge between raw audio and machine learning
- The model encodes features, then decodes probabilities into a word sequence
- The transcript is passed into downstream AI modules (LLMs, agents, classifiers)

02

Feature Extraction Deep Dive

From raw audio to ML-ready representations — step by step

Step 1 — Audio Acquisition & Preprocessing

STEP 1/6

Digitization

Mic converts pressure waves to discrete samples.
Common: 16 kHz (speech), 44.1 kHz (music).

Normalization

Amplitude scaled so quiet and loud recordings are treated equally by the model.

Noise Reduction

Background noise (fans, traffic) suppressed to improve signal quality before extraction.

Silence Detection

Non-speech segments identified and trimmed to save compute and reduce confusion.

Pre-emphasis

High-pass filter boosts high frequencies — balances spectrum for voiced sounds.

Resampling

Audio resampled to fixed rate (e.g. 16 kHz) so inputs match the model's expected format.

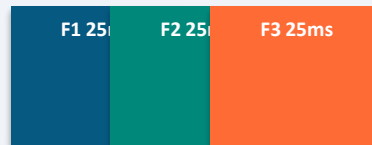
Step 2 — Framing & Windowing

STEP 2/6

Why Framing?

- Speech changes constantly — not stationary over long durations
- Cannot apply a single Fourier transform to a whole utterance
- Solution: cut signal into short overlapping windows (frames)
- Each frame captures a local acoustic snapshot of ~25 ms
- Frames overlap by 15 ms (stride=10 ms) to preserve boundary info

Overlapping Frames on Waveform



← 10 ms stride between frames →

Overlapping ensures no phoneme info is lost at frame boundaries.

Why Windowing?

- Raw frame edges cause abrupt jumps — FFT interprets these as spectral leakage
- Windowing (Hamming/Hann) tapers the frame smoothly to zero at both edges
- Result: cleaner, more accurate frequency estimates per frame

Step 3 — Fourier Transform & Spectrogram

STEP 3/6

What the Fourier Transform does

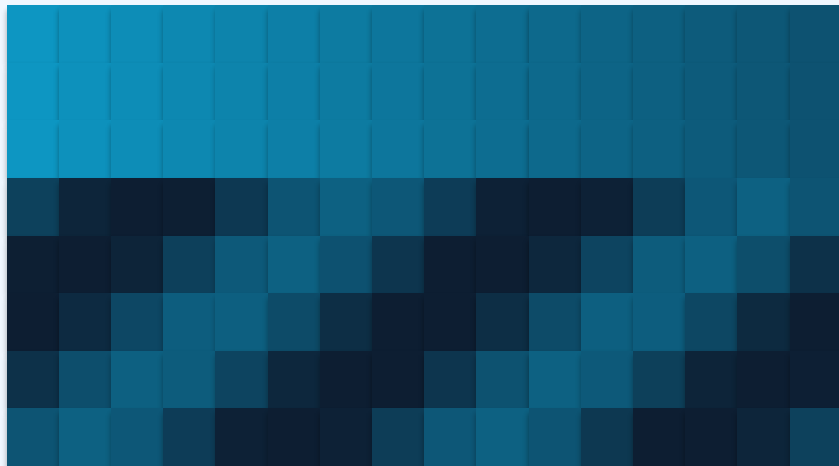
- Each frame is a short audio segment in the time domain
- The FFT reveals which frequencies are present and how strong each is
- Output: a frequency spectrum — energy per frequency bin
- Repeating across all frames builds a 2D time-frequency map: the spectrogram
- Voiced sounds (vowels) show harmonic patterns; consonants look different
- The spectrogram is the direct input representation before Mel filtering

Windowed Frame (time domain)

→
FFT
→

Frequency Spectrum

→ Repeat for all frames = Spectrogram
Spectrogram (simulated heatmap)



Time →

Step 4 — Mel Scale & Mel Filterbanks

STEP 4/6

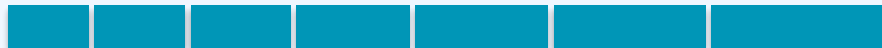
The Mel Scale — Perceptual Frequency

- Human hearing is more sensitive to pitch changes at low frequencies
- Below ~1 kHz: we detect even small pitch changes easily
- Above ~4 kHz: large frequency changes sound small to us
- Mel scale is a nonlinear Hz mapping matching human perception
- ASR uses Mel-spaced filters instead of evenly-spaced linear bins

Mel Filterbank — How It Works

- A bank of ~26–80 triangular filters across the Mel frequency axis
- Each filter sums the spectral energy in its frequency band
- Output: one energy value per filter — a compact MFBE vector
- Filterbank = perceptual compression of the raw spectrum

Filter density Low Hz —————> High Hz



More filters where the ear is sensitive — less where it is not

Before log: [0, 2000, 150000, 80, 5000]

Huge range — hard for neural networks to calibrate

Spectrum

× Mel Filters

Sum Energy

MFBE Vector

After log: $[-\infty, 7.6, 11.9, 4.4, 8.5]$

Compact, stable range — neural networks train well on this

Step 5 — Log Compression (Log-Mel Spectrogram)

STEP 5/6

Why take the logarithm?

- Mel filterbank energies span a huge dynamic range (e.g., 0 to 150,000)
- Neural network weights work poorly when inputs span many orders of magnitude
- The logarithm compresses this range — matching how human loudness perception works
- $\log(0) = -\infty$, so a small floor (e.g., $1e-10$) is added before applying the log
- This produces the Log-Mel Spectrogram — primary input for modern ASR models
- Whisper, wav2vec 2.0, and Conformer all use 80-bin log-Mel spectrograms as input

MFBE

+ floor

$\log()$

Log-Mel Spec.

Before log: [0, 2000, 150000, 80, 5000]

Wide range — hard for model weights to calibrate

After log: $[-\infty, 7.6, 11.9, 4.4, 8.5]$

Compact, stable range — neural networks train well on this

Step 6 (Optional Classical Path) — MFCC

STEP 6/6

Mel-Frequency Cepstral Coefficients (MFCCs)

- MFCC adds one more step after log-Mel: a Discrete Cosine Transform (DCT)
- DCT decorrelates the filterbank energies — adjacent Mel bands are correlated, DCT separates them
- Output: compact vector of ~13–40 coefficients (the cepstrum) per frame
- Nearly uncorrelated coefficients — ideal for classical ML models that assume independence
- MFCCs dominated ASR for decades (GMM-HMM systems, speaker ID, keyword spotting)
- Still used in lightweight, resource-constrained embedded systems today

Log-Mel Spec.

DCT

Cepstrum (13–40 coeff.)

DCT converts log-Mel filter energies into compact, decorrelated coefficients

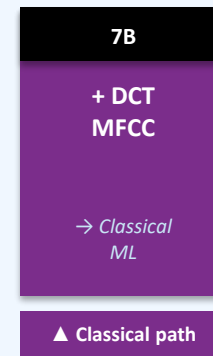
⚡ MFCC is a feature representation, not a recognition model. Recognizing speech is a separate step.

Feature Showdown — MFCC vs Log-Mel Spectrogram

Dimension	MFCC	Log-Mel Spectrogram
Extra step	DCT applied after log-Mel	No DCT — direct log-Mel output
Output size	13–40 coefficients per frame	26–128 frequency bins per frame
Correlation	Decorrelated — independent	Correlated — adjacent bands overlap
Information	Compact; some spectral detail lost	Richer — preserves full structure
Best for	Classical ML (GMM-HMM, SVM, kNN)	Deep learning (CNNs, Transformers)
Used by	Traditional ASR, speaker ID	Whisper, wav2vec 2.0, Conformer
Era	1980s–2010s dominant	2015–present dominant

Key insight: MFCC → Log-Mel mirrors the shift from feature engineering to learned representation — let the model learn from richer input.

Feature Extraction — Complete Flow Recap



03

Acoustic Modeling & Decoding

From features to transcript — how models predict text from sound

From Features to Hidden Representations — Acoustic Modeling

What the acoustic model does

- Feature sequences (log-Mel frames) are passed into a neural encoder that processes them over time
- The encoder builds contextual representations — each frame is seen in context of surrounding frames
- Modern encoders use Transformer self-attention, CNNs, or a combination (Conformer architecture)
- Encoded representations capture phonetic and acoustic patterns that predict speech tokens
- This is fundamentally a sequence-to-sequence learning problem

Transformer Encoder

- Multi-head self-attention across all frames
- Positional encoding for time order
- Used in: Whisper, wav2vec 2.0

CNN Front-end

- Local feature extraction from spectrograms
- Reduces sequence length before transformer
- Used in: Conformer, ContextNet

Conformer Hybrid

- CNN + Transformer combination
- Best of local + global context
- State-of-the-art for streaming ASR

Traditional ASR Pipeline vs End-to-End Models

Traditional Pipeline

Acoustic Model

GMM or HMM maps frames to phoneme probabilities

Pronunciation Dict

Phoneme sequences mapped to words

Language Model

Word sequences scored for fluency

Decoder

Finds most likely word sequence (beam search)

Each module trained independently → complex, brittle pipeline

End-to-End Model

Single Learned Model

Directly maps audio features to text output

Encoder-Decoder

Encoder processes audio; decoder generates tokens

Implicit Language Model

Language knowledge baked into model weights

Simpler API

Input: audio chunks → Output: text transcript

Simpler model — still needs preprocessing & integration layer

04

Pre-trained Models & AI Integration

Using speech perception as a reusable module in larger systems

Using Pre-trained Speech Models

Why pre-trained models change everything

- Training high-quality ASR from scratch requires thousands of hours of labeled audio — impractical
- Pre-trained models have already learned acoustic and linguistic patterns from massive datasets
- Can be used directly (zero-shot) or fine-tuned on a target domain with far less data
- Common capabilities: transcription, language ID, word-level timestamps, translation

Whisper (OpenAI)

680K hrs multilingual

Zero-shot transcription + translation;
99 languages

wav2vec 2.0 (Meta)

960 hrs + self-supervised

Self-supervised pre-training; fine-tune with <10 min labeled audio

Conformer (Google)

LibriSpeech + large corpora

CNN+Transformer; state-of-the-art on benchmarks

MMS (Meta)

500K hrs / 1100+ langs

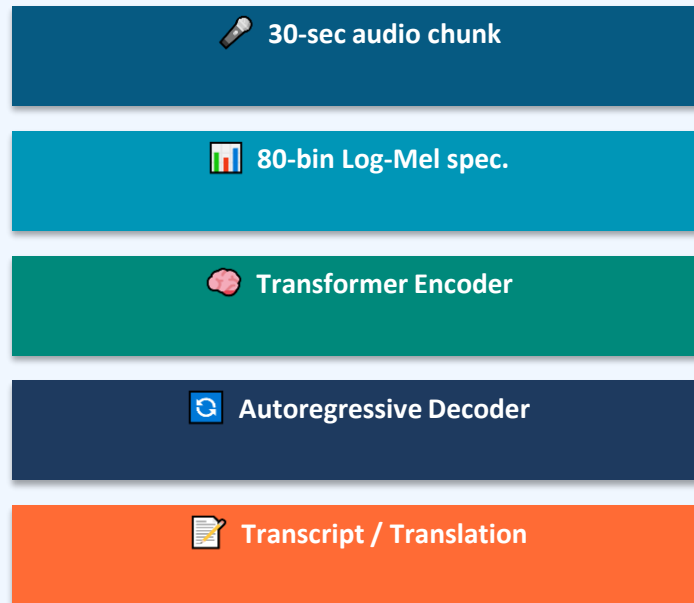
Massively multilingual; low-resource language focus

Whisper Deep Dive — Architecture & Capabilities

Whisper — Key Facts

Training data	680,000 hrs multilingual audio
Architecture	Encoder-Decoder Transformer
Input	80-bin Log-Mel, 30-sec chunks
Output	Transcript / Translation / Language ID
Languages	99 languages zero-shot
Model sizes	Tiny → Large-v3 (speed vs accuracy)

Whisper Processing Pipeline



Whisper connects every concept from this lecture into one production model

Streaming & Low-Latency Speech Recognition

Why streaming ASR is different

- Batch ASR waits for complete audio before transcribing — too slow for real-time interaction
- Streaming ASR outputs text as audio arrives — essential for voice assistants and live captioning
- Challenge: model sees only past frames, not future ones — less context = harder predictions
- Solutions: chunk-based processing, CTC decoding, lookahead windows, RNN-T architecture

Batch ASR

Wait for full audio → process → transcript.
High accuracy, high latency (several seconds).

Chunk Streaming

Process fixed-size chunks (e.g. 1 sec).
Moderate latency with small accuracy trade-off.

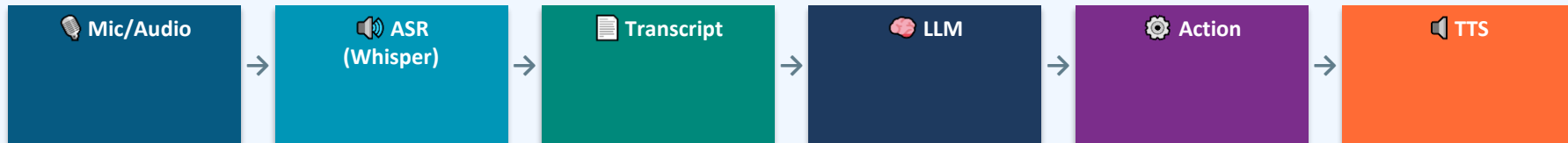
True Streaming

Frame-by-frame decoding. Minimal latency.
Used in Google Live Transcribe.

Integrating ASR into AI Pipelines

ASR as a perception module — design patterns

- The transcript produced by ASR is the entry point for all downstream reasoning
- ASR integrates with LLMs via tool calls, function calling, or direct prompt injection
- Pipelines may chain: ASR → NLU → Intent → Action → TTS for full voice loops
- Timestamps enable speaker diarization, meeting summarization, and search-by-phrases



Common Integration Patterns:

- Voice agent: ASR → LLM → tool use → TTS response
- Meeting assistant: ASR with diarization → summarization → action items
- Live captioning: streaming ASR → real-time subtitle overlay
- Command interface: keyword ASR → intent classifier → system action

Design Trade-offs in Speech Systems

Key decisions when building an ASR-powered system

Accuracy vs Latency

Larger models are more accurate but slower. Streaming sacrifices some accuracy for speed. Choose based on your use case tolerance.

On-device vs Cloud

On-device (e.g. Whisper Tiny) preserves privacy and works offline. Cloud models offer higher accuracy with network dependency.

General vs Domain-Specific

General models (Whisper) work zero-shot. Fine-tuned models outperform on specific vocabulary (medical, legal, technical).

Mono vs Diarized

Single-channel ASR gives one transcript. Diarization identifies who speaks when — needed for meetings, call centers.

Challenges & Failure Modes in ASR

Where ASR systems break down — and why

Accents & Dialects

Models trained on standard speech underperform on regional accents, non-native speakers, and code-switching.

Background Noise

Cafés, traffic, and fan noise degrade accuracy significantly — noise robustness requires separate training.

Domain Vocabulary

Medical, legal, or technical jargon unseen during training leads to substitution errors (homophones, rare words).

Overlapping Speech

Multiple simultaneous speakers confuse the acoustic model — diarization is a separate, harder problem.

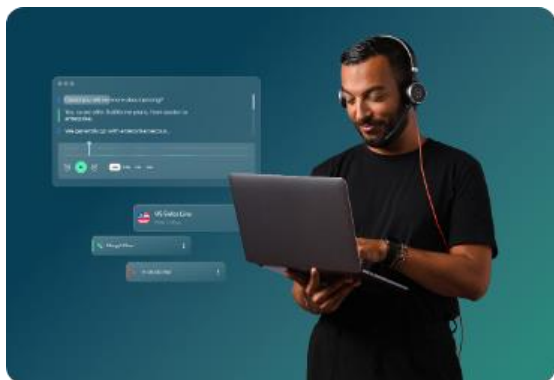
Low-Resource Languages

Most high-accuracy models are English-centric. Smaller languages lack sufficient training data.

Hallucination

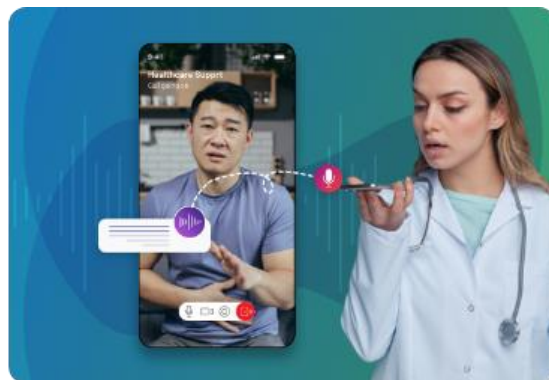
Large seq2seq models (like Whisper) sometimes generate plausible but wrong text, especially on silence or noise.

Perception in the Wild



Customer Service

Automated QA and Sentiment Analysis in call centers.



Medical AI

Hands-free clinical documentation for physicians.



Accessibility

Real-time captioning for live events and video meetings.

Key Takeaways — Week 8

① Speech = Structured Data

ASR converts raw audio into text — giving AI systems a natural language understanding entry point.

② Feature Extraction is Central

The pipeline from waveform → frames → FFT → Mel filters → log compression is the foundation of all modern ASR.

③ Log-Mel > MFCC for Deep Learning

Modern models consume log-Mel spectrograms directly; MFCC remains relevant for classical/embedded systems.

④ Pre-trained Models = Practical Path

Whisper and wav2vec 2.0 make high-quality ASR accessible — zero-shot or fine-tuned with minimal data.

ASR is the ear of the AI system — master it and every voice-driven application becomes possible.